

Developing a Pattern Discovery Method in Time Series Data and Its GPU Acceleration

Huanzhou Zhu, Zhuoer Gu, Haiming Zhao, Keyang Chen, Chang-Tsun Li, and Ligang He*

Abstract: The Dynamic Time Warping (DTW) algorithm is widely used in finding the global alignment of time series. Many time series data mining and analytical problems can be solved by the DTW algorithm. However, using the DTW algorithm to find similar subsequences is computationally expensive or unable to perform accurate analysis. Hence, in the literature, the parallelisation technique is used to speed up the DTW algorithm. However, due to the nature of DTW algorithm, parallelizing this algorithm remains an open challenge. In this paper, we first propose a novel method that finds the similar local subsequence. Our algorithm first searches for the possible start positions of subsequence, and then finds the best-matching alignment from these positions. Moreover, we parallelize the proposed algorithm on GPUs using CUDA and further propose an optimization technique to improve the performance of our parallelization implementation on GPU. We conducted the extensive experiments to evaluate the proposed method. Experimental results demonstrate that the proposed algorithm is able to discover time series subsequences efficiently and that the proposed GPU-based parallelization technique can further speedup the processing.

Key words: dynamic time warping; time series data; data mining; pattern discovery; GPGPU; parallel processing

1 Introduction

Time series data is a series of values sampled at time intervals^[1]. Time series data captures a lot of information including inter-patterns, trends, and correlations. Also, time series data usually contains a high volume of numerical data and a time dimension. Many different techniques have been developed to model, compare, and predict time series data.

Due to the ubiquity of time series data, many real life

data mining tasks can be modelled as mining time series data. These jobs often relate to finding known patterns, discovering unknown patterns, and locating common subsequences in time series data^[2–5]. Locating common subsequence or sequence matching is an important domain in time series data analysis.

As suggested in Refs. [6,7], many applications can be solved by finding the common subsequence pairs in two time series data, for example, finding the association rules in time series data^[8,9], classification algorithms that are based on building typical prototypes of each class^[10,11], anomaly detection^[12], and finding periodic patterns^[13].

Dynamic Time Warping (DTW)^[14] is a well-known algorithm for comparing similarities between two time series data. DTW algorithm is designed for finding the global sequence alignment in speech recognition. To find local alignments, previous researchers focus on finding subsequence within a longer sequence

• Huanzhou Zhu, Zhuoer Gu, Haiming Zhao, and Ligang He are with Department of Computer Science, University of Warwick, Coventry, UK. E-mail: ligang.he@warwick.ac.uk.

• Keyang Chen is with School of Computer Science and Telecommunications Engineering, Jiangsu University, Zhenjiang 212013, China.

• Chang-Tsun Li is with School of Computing and Mathematics, Charles Sturt University, Wagga Wagga, Australia.

* To whom correspondence should be addressed.

Manuscript received: 2018-01-03; accepted: 2018-03-02

that match a shorter query sequence (Subsequence DTW)^[15–17].

However, DTW algorithm suffers from high computational complexity. A straightforward implementation of DTW is quadratic in time and space. Using DTW to find local similarity between two time series normally requires applying the DTW algorithm to input data multiple times^[18–20]. Therefore, it is necessary to improve the performance of the DTW algorithm.

Based on the above considerations, many efforts are made to develop methods and techniques that execute the DTW algorithm in parallel to reduce the execution time. Recently, with the development of general purpose GPUs and programming environment, much research is conducted to study how to execute the DTW algorithm on GPUs^[21–23].

However, due to the data dependency, parallelizing DTW algorithm on GPU is challenging and suffers from low parallelization degree^[21]. Therefore, in this paper, we propose a novel method to parallelize the DTW algorithm on GPU. Based on dynamic programming, we first propose a new algorithm of discovering similar local subsequences between time series sequences efficiently. Then we parallelize the proposed algorithm on GPU and use the diagonal order technique to optimize the memory access pattern in GPU.

This paper is organized as follows: Sections 2 and 3 provide the related work and the background in time series data analysis. Section 4 proposes the design of the subsequence matching method. Section 5 presents the GPU implementation of the parallelized algorithm. We present the evaluation and experimental results in Section 6. Section 7 concludes this paper.

2 Related Work

Many researchers have conducted work in finding subsequences in time series data. In Ref. [6], Lin et al. developed a symbolic representation-based local subsequence matching algorithm to find the k -th most similar subsequence, but the algorithm requires a user-input length of the subsequence. A probabilistic method for subsequence matching is proposed in Ref. [7]. The method tries to match arbitrary subsequence of one time series to an arbitrary subsequence of another time series and analyze the statistical pattern of a result matrix to locate the position of the sequence. This method also requires the length of the subsequence

to be known in advance. In Ref. [24], a local subsequence matching algorithm for multi-dimensional data is proposed. Reference [25] presents a method of discovering exact subsequence (the length and the shape of the subsequence are known in advance). The algorithm provides a feasible solution in finding exact subsequence from short time series in a reasonable time. However, the time complexity of the method is still very high, which makes it not practical to work with long time series.

Toyoda et al.^[26] proposed a novel method to discover similar subsequence in time series. The algorithm creates two matrices, score matrix and position matrix. The score matrix locates the end position of the found subsequence, and the position matrix uses the information collected from the process of computing score matrix to calculate the start position of the subsequence. This method can find subsequences at any length and of any shape without producing meaningless subsequences and reduces required time and space complexity significantly in the meantime. Moreover, this method only needs three parameters and does not require prior knowledge of the subsequences to be found. However, this algorithm is still lack of scalability thanks to the exponential increase of matrix size with the accumulation of time series.

The work in Ref. [27] exploits the possibility of using CPU clusters to speed up DTW. The method assigns subsequences starting from different positions of the time series to different processors, and each processor performs the classic DTW algorithm. The method proposed in Ref. [28] explores the use of the multi-core processor to parallelize the DTW algorithm. This work separates different patterns into different cores, and each subsequence will be assigned to different cores, where it is compared with different patterns naively. In these two parallel implementations, because a subsequence consists of several hundred or thousands of tuples, the data transferring becomes the bottleneck of the system.

In Ref. [23], a GPU implementation of DTW algorithm is proposed. In this work, multi-threads are used to generate the warping matrix in parallel, but the path search is performed serially. This implementation exploits the partial fine-grained parallelism of DTW itself. Because it separates the matrix generation phase and the path search phase into two kernels, it needs to store and transfer the whole warping matrix for each DTW calculation, which is also a heavy burden in

the whole system. In Ref. [22], the authors claimed that they are “the first to present hardware acceleration techniques for similarity search in streams under the DTW measure”. Their GPU implementation is similar to Ref. [27].

3 Background

In this section we review the concepts of time series and DTW distance^[14] as well as some background knowledge for the ease of further discussion.

3.1 Time series

A time series can be represented as $T = (t_1, t_2, \dots, t_n)$ which denotes ordered values in a time series of n samples^[14]. The sequences are measured within fixed time intervals. A subsequence S with length k of T is denoted as $S = (t_i, t_{i+1}, \dots, t_{i+k})$, where $1 \leq i \leq n - k$.

Given sequence $X = (x_1, x_2, \dots, x_n)$ of length n and sequence $Y = (y_1, y_2, \dots, y_m)$ of length m , the similar subsequences of X and Y are S_x and S_y that satisfy the pre-defined similarity measurement condition. In this paper, we aim to find all similar subsequences S_x and S_y from two time series.

3.2 Classic DTW algorithm

Since the classic DTW algorithm serves as the foundation of our proposed algorithm in this paper, we introduce the algorithm in this section briefly.

The classic DTW algorithm is based on dynamic programming technique, which stores the previous computed results in a matrix so that the later calculation can use these results directly without re-computing them. Each cell in the matrix corresponds to an alignment of a point in both time series in each dimension, e.g. cell (i, j) corresponds to the alignment of i -th and j -th time stamp in each time series. The value represents the DTW distance between the time series starting from the first data point up to the corresponding alignment point.

Therefore, given two time series, $X = \{x_1, x_2, \dots, x_m\}$ of length m and $Y = \{y_1, y_2, \dots, y_n\}$ of length n , the DTW distance is calculated by following equations:

$$\begin{aligned} d(0, 0) &= 0, \\ d(i, 0) &= d(0, j) = \infty, \\ d(i, j) &= \|x_i - y_j\| + \min(d(x_{i-1}, y_j), \\ &\quad d(x_i, y_{j-1}), d(x_{i-1}, y_{j-1})), \end{aligned}$$

$$D(X, Y) = d(m, n) \quad (1)$$

where $i = 1, 2, \dots, m$ and $j = 1, 2, \dots, n$ and $D(X, Y)$ is the DTW distance between X and Y . $\|x_i - y_j\|$ is the distance between x_i and y_j , which can be $(x_i - y_j)^2$ or simply $|x_i - y_j|$, or other possible measures.

3.3 Distance measure

Given two time series X and Y , the distance between any two points x_i and y_j from these two time series can be computed by *cost* function. There are many choice of the *cost* function, the most intuitive way is to use Manhattan distance, which is the absolute difference between x_i and y_j :

$$\text{Manhattan_distance}(x_i, y_j) = \text{abs}(x_i - y_j) \quad (2)$$

Euclidean distance is a well known method in measuring distance, which is the root of square difference:

$$\text{Euclidean_distance}(x_i, y_j) = \sqrt{(x_i - y_j)^2} \quad (3)$$

Another widely used measurement is the normalized distance, which measures the difference between two points in the sequence with regard to the range of the sequences. That is, the difference between the maximum and minimum values in the sequences: $R_x = |X|$, $R_y = |Y|$. The normalized Manhattan distance can be computed with the following formula:

$$\text{Manhattan_distance_norm}(x_i, y_j) = \frac{\text{abs}(x_i - y_j) \times 2}{R_x + R_y} \quad (4)$$

3.4 GPGPU and CUDA

Using GPU as a general computing unit has attracted considerable attention. GPU provides massive parallel processing power. As the host for the GPU device, CPU organizes and invokes the kernel functions that execute on GPU. As shown in Fig. 1a, a GPU device consists of a number of Streaming Multiprocessors (SMs), each

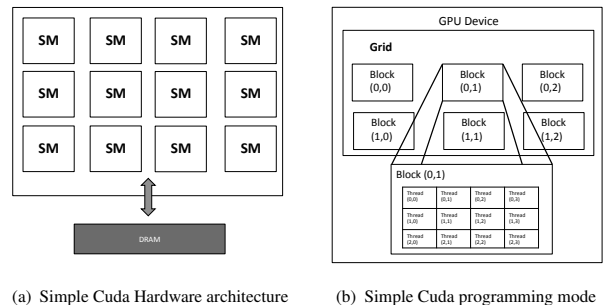


Fig. 1 CUDA programming model.

comprising simple processing engines, called CUDA cores in the NVIDIA terminology^[29]. Each SM has its own shared memory, which is equally accessible by all CUDA cores in the SM. At any given cycle, the CUDA cores in an SM execute the same instruction on different data items. SMs communicate with each other through the global memory of GPU.

From the programmers' perspective, the CUDA model^[29] is a collection of threads running in parallel. A warp is a collection of threads that can run simultaneously on an SM. The warp size is fixed for a specific GPU. The programmer decides the number of blocks and threads to be executed. If the number of threads is more than the warp size, they are time-shared internally on the SM. A collection of threads (called a block) runs on a multiprocessor at a given time. Multiple blocks can be assigned to a single multiprocessor and their execution is time-shared. A single execution generates a number of blocks. A collection of all blocks in a single execution is called a grid (Fig. 1b). All threads of a block executing on a single multiprocessor divide its resources equally amongst themselves. Each thread and block is given a unique ID that can be accessed within the thread during its execution. Each thread executes a single instruction set called the kernel.

4 Algorithm for Finding All Similar Subsequences

In this section, we propose a novel subsequence pattern mining algorithm. The objective of our algorithm is to discover all similar subsequences between two time series. Our proposed algorithm is based on the classical dynamic time warping algorithm and is adapted so that it can capture all subsequence patterns. We first introduce an intuitive strategy to find all similar subsequences. Then we propose two novel pattern mining algorithms.

4.1 Naive subsequence matching

The naive approach is to break the time series X into all possible subsequences, and compare them with all the subsequences of Y using the DTW algorithm and select the best matches based on the defined matching condition. That is, for all $X_s = (x_i, x_{i+1}, \dots, x_{i+k})$, where $1 \leq i \leq n - k$, compute the DTW distance against all $Y_s = (y_j, y_{j+1}, \dots, y_{j+l})$, where $1 \leq j \leq m - l$, n and m are the length of X and Y , respectively.

This approach will compute m DTW matrix for every

timestamp in X , this is because it needs to examine all possibilities. Therefore, the complexity of this method is in quadratic time: $O((1 + 2 + \dots + n)(1 + 2 + \dots + m)) = O(n^2 m^2)$.

Another brute force method is to use the shotgun window and try to match every possible window between the two sequences. For each window in the first sequence, compute DTW for all windows in the second sequence, and select the pairs with the best match, that is, with the lowest DTW cost. Figure 2 illustrates this approach. The complexity of the shotgun window approach is $O(m^2 n^2 / w^2)$, which only reduces the complexity by w^2 times. These naive approaches are not suitable for large-scale datasets, as the time complexity increases quadratically.

Both algorithms presented in this section need to examine all possible combinations between different time series. Hence, the time complexity of both algorithms is high, which motivates us to design more efficient algorithms to find all similar patterns between two time series.

4.2 Proposed algorithm

The intuitive algorithms work by applying the DTW algorithm on all subsequences in two time series. Recall that the DTW algorithm works by adding the distance between two data points from two time series together. Therefore, we argue that if the distance between two data points is large, it is less likely that it will lead to a similar pattern.

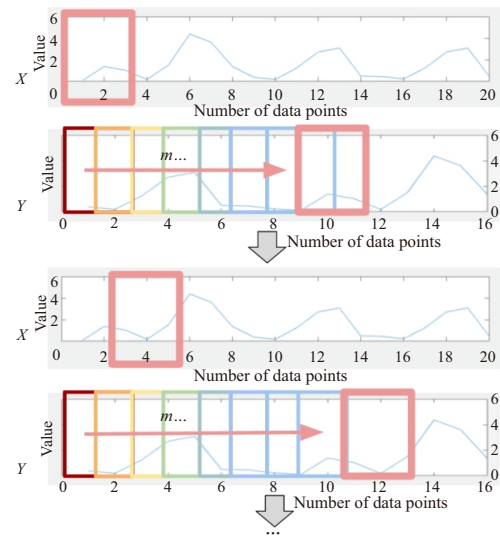


Fig. 2 The brute force shotgun windows approach. In these figures, the x-axis represents the length of input time series data (number of data points), the y-axis represents the value of each data point.

To verify our hypothesis, we conducted the following experiment. For two time series X and Y , we use the cost function to compute the distance between every data point and store the results in a matrix. We call this matrix the cross-distance matrix between two sequences, the value stored in each cell represents the distance between data points x_i and y_j . We visualize this matrix in Fig. 3. In this figure, the darker area represents small values, and they correspond to the data points that have a shorter distance. From Fig. 4, we can see that the alignments lie in the low-cost areas in the matrix, where the colour is the darkest in the visualization.

Based on the finding above, we modify the classic DTW algorithm so that it will only search for similar

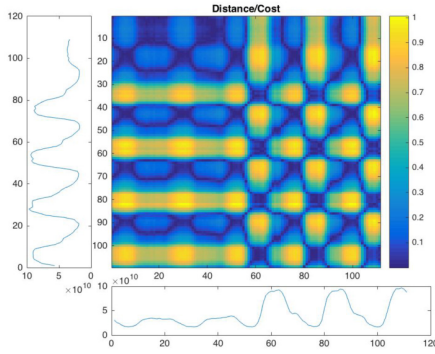


Fig. 3 Normalized Manhattan cost matrix. In two input data frames, the x -axis represents the length of input time series data (number of data points), the y -axis represents the value of each data point. The x - and y -axes in the Distance/Cost matrix represent the distance between two corresponding data points.

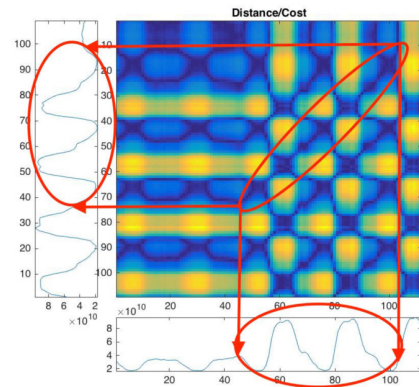


Fig. 4 Annotated cost matrix. In two input data frames, the x -axis represents the length of input time series data (number of data points), the y -axis represents the value of each data point. The x - and y -axes in the Distance/Cost matrix represent the distance between two corresponding data points.

subsequences from the low-cost values cells.

In this algorithm, we first determine the low-cost values of the input time series. To achieve this, we introduce the cost threshold t , which is defined as the minimum average cost for each matching data points. This threshold t is used to determine whether to start or continue a DTW calculation. The procedure should accept the alignment (i, j) ($1 \leq i \leq N$ and $1 \leq j \leq M$) if $\frac{dtwDistance}{dtwLength} < t$, where

$$dtwDistance = \sum_{i,j \in warpingPath} cost(x_i, y_j) \quad (5)$$

and $dtwLength$ is the length of the warping path.

If the DTW distance is smaller than this threshold, the path is considered as a possible match and will be included in the further computation.

To avoid over-warping in the warping path during the searching, the window constraint is imposed on the computation. In this method, we define an offset threshold o to control the length of the warping window, as illustrated in Fig. 5. In this figure, the warping window is defined by the red region, and the threshold o is the length of the warping window offset from the starting position. With the offset threshold, the computation of DTW will stop when the computation reaches the warping window boundary, and therefore, the warping path will not exceed the length of the warping window.

In some cases, the matching subsequences founded by the DTW algorithm may be too short to be considered as a valid match. To tackle this problem, we define the window threshold w to filter out the subsequences that are too short. The window threshold w is defined as the minimum alignment length of a sequence. As shown in Fig. 6, suppose the red line is a warping path, the length of the warping path must be longer than w for the subsequence to be considered.

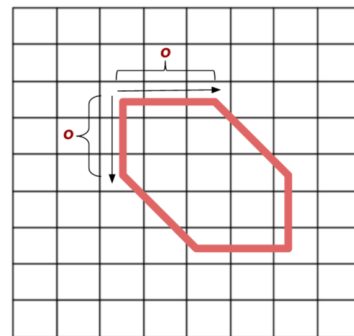


Fig. 5 Offset threshold and warping window.

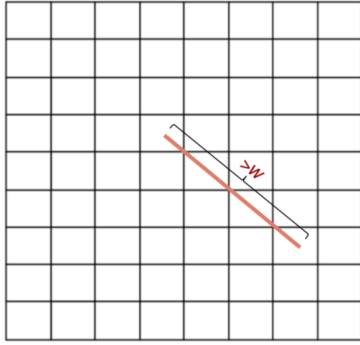


Fig. 6 Window threshold: Warping path length.

4.3 Formal definition of the algorithm

Based on the discussion in the previous sections, we first summarize the problem as the following:

Given sequence $X = (x_1, x_2, \dots, x_n)$ of length n and sequence $Y = (y_1, y_2, \dots, y_m)$ of length m , the algorithm searches for all subsequence $S = s_i, s_{i+1}, \dots, s_{i+k}$, where S is a set of all subsequence of both X and Y that satisfy the following conditions:

- (1) The cost of the alignment per data point is less than t , that is, $\frac{d}{l} \leq t$, where d is total cost of the alignment, i.e., the sum of the cost of every alignment point in the warping path of the matching subsequence; l is the length of the warping path, i.e., the total number of the alignment points in the warping path.
- (2) The offset of the warping path from the perfect diagonal line must not exceed o data points, as explained in Section 4.2.
- (3) The length of the warping path must be longer than the window threshold w , as explained in Section 4.2. If the found match is too short, i.e., smaller than w , this match will not be included in the final result.

The proposed algorithm is based on the dynamic programming, which updates a matrix of size $m \times n$, where m and n are the length of two time series X and Y , respectively. In our algorithm design, we first create the following matrix to store the intermediate and final results:

- D : This matrix is used to hold the cumulative time warping cost, which takes $O(mn)$ space.
- C : This matrix is used to store the distance between two data points. It is worth noting that this matrix is optional, since the cost between two data points can be computed on the fly while updating D .

- L : This matrix stores the current length of the warping path of the corresponding alignment. We increment the length of warping path while calculating the cumulated cost and finding the matched subsequences. We use the value stored in this matrix to determine if the match can be included in the final results.
- R : This matrix is the region-marking matrix that stores the position of the start point and end point of the matched subsequences, which will be used in the lookup and the trace-back processes.
- P : This matrix is used to store the direction of previous alignment point of the warping path, which is used in trace back to find the warping path.

The proposed algorithm is performed in three steps: initialization, updating the matrix, and finding the path (trace back), which are detailed as follows.

(1) Initialization: The algorithm initializes all the matrices to zero and initializes cumulative cost matrix D to infinity. Therefore, during the update, the infinity will be ignored in the minimum comparison. If required, the cost matrix C can be computed.

(2) Subsequence DTW calculation: This step performs the modified DTW algorithm. The algorithm starts from the top left position of the matrices, and iterates these matrices in the diagonal order.

If the current cell in the matrix is not part of any warping path and the cost of the current alignment is smaller than the cost threshold t , we mark this position as the start point in the region marking matrix R . Then we set the length of the warping path from the current cell in length matrix L as 1, update the cumulative cost to the current cost, and update other matrices accordingly. Otherwise, if the cost of current alignment is not smaller than threshold t , then we move on to the next position.

If the cell being examined belongs to a warping path, then we select the cell with the smallest cumulative cost from the warping path. Next, we compute the average cost by adding the cost of the current position to the cumulative cost and divide this value by the total length. If the sum of the average cost and the cost of the current cell is smaller than threshold t , we include this cell as part of the matching subsequence from the cell with the smallest cumulative cost in the warping path, and update L , D , R , P , respectively.

If all the elements in the warping path are not part of any matching subsequences, we first determine if the

cost of this cell is smaller than t . If it is, then we start a new subsequence from the current cell and update all the matrices accordingly.

(3) Finding paths: In this step the algorithm simply uses matrixes R and L to find all the matched subsequences. The algorithm also uses threshold w to check if the length of the warping path is valid, then use P to trace back the warping path and return the results.

We outlined the above procedure in Algorithms 1 and 2.

4.4 Further improvement

The algorithm described in the last section uses the minimum cumulative cost in matrix D to select the best cell for the DTW path. This approach requires

Algorithm 1: DTW-Based Subsequence Pattern Mining.

Input: Time series X and Y , threshold t and o
Output: Matrices with subsequence matching info: D, L, R, P

```

1 Initialisation:  $D$  to  $inf$ ;
2  $L, R$  and  $P$  to 0;
3 calculate cost matrix  $C$ ;

/* begin dynamic programming */
4 for  $i = 1 : \text{length}(X)$  do
5   for  $j = 1 : \text{length}(Y)$  do
6     if  $i == 1$  and  $j == 1$  then
7        $minpre = 0$ ;
8        $mini = i; minj = j$ ;
9     else
10       $minpre = \min(D[i-1, j-1], D[i-1, j], D[i, j-1])$ ;
11       $minpre = (minpre == inf) ? 0 : minpre$ ;
12      // set back to 0 if all previous cells are
13      // still in init state
14       $mini, minj = \text{argmin}(D[i-1, j-1], D[i-1, j], D[i, j-1])$ ;
15    end
16     $dtwm = (minpre + C[i, j]) / (L[mini, minj] + 1)$ ;
17    if  $dtwm \leq t$  and  $L[mini, minj] == 0$  then
18      /* start a new region */
19       $D[i, j] = C[i, j]$ ; // update dtw distance
20       $L[i, j] = 1$ ; // update path length
21       $R[i, j] = (i, j, i, j)$ ; // update start and end
22      cell
23    else if  $dtwm \leq t$  then
24      /* continue from previous region */
25       $(si, sj, li, lj) = R[mini, minj]$ ; // find si, ji as the
26      // region start cell
27      // if current not diverge too much from
28      // offset
29      if  $abs((i - si) - (j - sj)) < o$  then
30         $D[i, j] = minpre + C[i, j]$ ; // update dtw
31        distance
32         $L[i, j] = L[mini, minj] + 1$ ; // update
33        path length
34         $R[i, j] = (si, sj, i, 0)$ ; // update start
35        cell
36         $P[i, j] = (mini, minj)$ ; // update path
37      if  $i > li$  and  $j > lj$  then
38        // update end cell if furthest away
39         $R[si, sj] = (si, sj, i, j)$ ;
40      end
41    end
42  end
43 end

```

Algorithm 2: Find Paths.

Input: Matrices with subsequence matching info: L, R, P , threshold o
Output: Matrices Path marking: OP

```

1 Initialisation:  $OP$  to 0;
2 for  $i = 1 : \text{length}(X)$  do
3   for  $j = 1 : \text{length}(Y)$  do
4     // if this is the start
5     if  $L[i, j] == 1$  then
6        $li, lj = R[i, j]$ ; // find end position
7     if  $L[li, lj] > w$  then // if path length > w
8       while  $li > si$  and  $lj > sj$  do
9         mark  $OP[li, lj]$  as matching sequence path;
10         $(li, lj) = P[li, lj]$ ; // get previous
11        step
12      end
13      mark  $OP[li, lj]$  as matching sequence path;
14    end
15  end
16 end

```

the computation of the cumulative cost and the extra memory to store the results. Therefore, in this section, we propose a greedy algorithm to overcome this problem and further improve the performance.

The design of the greedy algorithm is based on the Markov assumption. That is, instead of using the cumulative cost, we only select the cell with the minimum cost from the warping path, and assume that the path from this cell leads to a path with the lowest cumulative cost. With this assumption in mind, we change line 10 and line 12 in Algorithm 1 to Eqs. (6) and (7), respectively, and remove lines 16 and 22.

$$minpre = \min(C[i, j-1], C[i-1, j], C[i-1, j-1]) \quad (6)$$

$$\min i, \min j = \text{argmin}([C[i, j-1], C[i-1, j], C[i-1, j-1]]) \quad (7)$$

However, compared to the original design, the greedy algorithm does not achieve significant improvement in terms of performance and space. This is due to the following reasons. First, since the original algorithm is based on dynamic programming, in every iteration the cumulative cost is computed based on the result produced in the last step. Therefore, computing the cumulative cost is not an expensive operation. Second, although the greedy algorithm does not require matrix D to store the cumulative cost, it still uses all the rest matrices in computation. Hence, the space complexity is not reduced significantly. In addition, the greedy algorithm cannot find the optimal path because it only uses the local optimal results in finding the subsequence. However, in the DTW algorithm, the local optimal result may not lead to the global optimal

solution.

5 GPU Acceleration

In this section, we discuss how to use CUDA to parallelize the algorithm presented in last section. We first introduce the parallelization strategy used in our design, and then present the CUDA implementation. Further, we present our method to optimize this GPU implementation for the GPU memory hierarchy.

5.1 Data dependencies

Recall that our algorithm uses the DTW algorithm as core to search for similar patterns. In DTW algorithm, however, the computation of each iteration depends on the result from the last step, which will limit the parallelization degree during the computation. Therefore, we first analyze the data dependencies in our algorithm and then design a strategy to improve the parallelization degree.

We show the data dependencies of the DTW computation in Fig. 7a. In the matrix, each cell represents the DTW distance between two data points. The computation of each cell depends on the results from left, top, and top-left cells. That is, for cell (i, j) in the matrix D , the value of $D[i, j]$ depends on the values in $D[i, j - 1]$, $D[i - 1, j]$, and $D[i - 1, j - 1]$.

Therefore, for any cell (i, j) in the matrix, all cells between $(0, 0)$ and (i, j) need to be computed before calculating the value of position (i, j) . In the classic DTW algorithm, this computation is carried out in the row or column order. The cells in the same row or column are computed in sequence. On the other hand, there is no data dependency between the diagonal cells, as illustrated in Fig. 7b. The cells connected by a red stripe have no data dependency. Therefore, these cells can be computed in parallel. The data dependency exists between two neighbouring red strips. Therefore, we can only compute the data within one red strip at time. We illustrate this in Fig. 7b with grey arrow. The arrow

indicates the data dependency and compute direction.

Based on the above discussion, we can summarize the parallel strategy as follows. The computation starts with the first cell, then updates the cells in the closest diagonal lines (the red stripe in Fig. 7b). Since these cells only depend on the first cell, they can be computed in parallel. Then the computation moves to the next diagonal line. By updating the matrix in this way, it is guaranteed that the data dependency is not broken and the reasonable degree of parallelism can be achieved.

5.2 Implementation of parallelization strategy

The previous section outlines a strategy to update the matrix in diagonal order. In this section, we describe how to implement this strategy in CUDA.

To implement this strategy on GPU, we need to consider how to map the CUDA threads to the data that need to be processed. Recall that the parallel diagonal computation model solves the data dependency issue by computing the matrix cells in diagonal order. Therefore, for each diagonal line, we can assign one thread to a cell within this line, and compute these cells in parallel.

An intuitive method to implement this strategy is to launch one kernel with enough threads, the number of threads within this kernel is larger or equal to the longest diagonal. In this design, the computation forms a loop within the kernel and each iteration processes one diagonal line. By implementing the algorithm in this way, we only need to launch one kernel, and therefore, reduce the overhead of multiple kernel launch.

In this implementation, we assign each column of the matrix to a thread first. Due to the data dependency, these threads are not processing each column simultaneously. Instead, each thread except the first one starts the processing when the previous thread finishes processing one cell in the matrix. By arranging the computation in this way, we can ensure that the data dependency is preserved and the data in the same diagonal line can be processed in parallel. This is because when the first thread processes the first cell in the matrix, the rest threads are waiting for this thread to finish. Once this computation is finished, the thread moves to the next item in the column, and its consecutive thread can start processing the first cell in the column assigned to it. These two threads can run in parallel because the cells involved in the computation can form a diagonal line in the matrix, and therefore, there are no dependency between them. However, as we mentioned in the last section, the results of the cells within a diagonal line depend on the cells

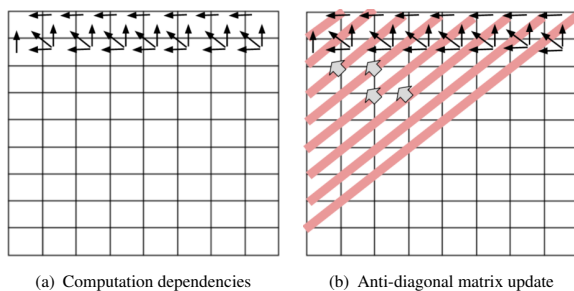


Fig. 7 Matrix update order.

in the previous diagonal line. Therefore, to enforce this dependency, we need to synchronize all threads involved in the computation along the diagonal line. We illustrate this design in Fig. 8. In this figure, the orange area represents the matrix that needs to be processed, the threads are represented by the red arrow, and the synchronization point is labelled by the red diagonal line.

From Fig. 8, we can see that the computation forms a parallelogram. The number of threads needed in the computation equals to the height of this parallelogram, which is n_y . And the number of iterations required equals to the base of the parallelogram, which is $n_x + n_y - 1$ number of rows. Therefore, the time complexity of this implementation is $O(n_x + n_y - 1)$, which is a significant reduction from $O(n_x \times n_y)$.

5.3 Memory access optimization

Coalesced access data in the global memory is critical to the performance of a GPU application. Non-coalesced memory access will lead to poor performance. In this section, we present how to enforce the coalesced memory access by enabling the consecutive threads to access consecutive memory address in our implementation.

Recall that our initial implementation processes the

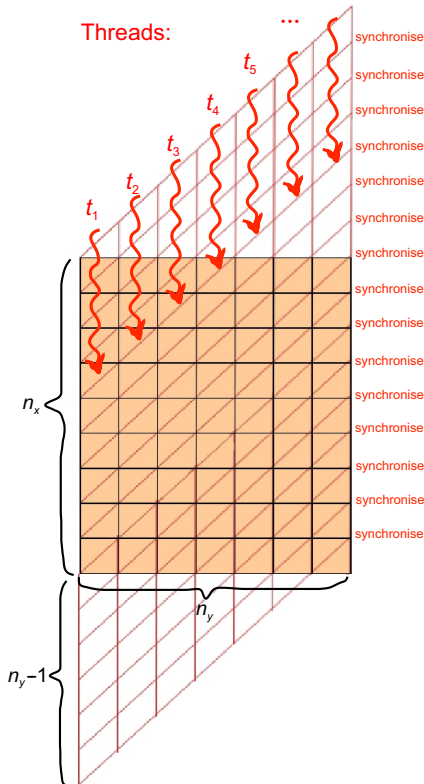


Fig. 8 Subsequence DTW matrix update threading.

data in the diagonal order. However, the data is stored as a matrix in the memory, and the elements on one diagonal line are not stored consecutively in the memory. Therefore, to achieve the coalesced access to the global memory, the data stored in the memory need to be rearranged so that the elements on the same diagonal line are stored consecutively in the GPU memory.

Intuitively, rearranging the matrix can be achieved by reallocating another 2D array. In this new array, the length of x -dimension is the length of the longest diagonal line in the original matrix, while the y -dimension is the number of the diagonal lines in the original matrix. In the new array, the diagonal lines from the original matrix are stored consecutively in the memory. We illustrate the array in Fig. 9.

However, as shown in Fig. 9, storing the diagonal line into the new matrix leads to the waste of memory space. To address this problem, instead of storing the diagonal line into a 2D array, we use a flat array to store the data from the diagonal line consecutively.

In addition, when calculating the warping boundary for the offset threshold, it is necessary to know the row and column positions of each element. Therefore, we create another matrix, namely index matrix I , to store the index of the flat array in the corresponding row and column positions.

The array indices can be determined through mathematical derivation, and be calculated in the initialization step in parallel. In the next section, we present how to derive these index numbers in the matrix.

5.4 Calculating diagonal index

We draw some small matrix and fill in the index number

0,0						
1,1	0,1					
2,1	1,2	0,2				
3,1	2,2	1,3	0,3			
4,1	3,2	2,3	1,4	0,4		
5,1	4,2	3,3	2,4	1,5	0,5	
6,1	5,2	4,3	3,4	2,5	1,6	0,6
7,1	6,2	5,3	4,4	3,5	2,6	1,7
8,1	7,2	6,3	5,4	4,5	3,6	2,7
8,2	7,3	6,4	5,5	4,6	3,7	
8,3	7,4	6,5	5,6	4,7		
8,4	7,5	6,6	5,7			
8,5	7,6	6,7				
8,6	7,7					
8,7						

(a) Original matrix

(b) Diagonal matrix

Fig. 9 Rearranging the matrix.

to study the relationship between the array index and the row and column (i, j) in the matrix. The indices in the upper left corner and bottom-right corner in the matrix should be calculated differently. This is due to the following reason. If the index is computed from left to right, then the upper left part starts from the left most column, while the bottom right part starts from the bottom most row. As shown in Figs. 10 and 11, these figures outlined two situations of the index number. The matrix represents the alignment of sequences X and Y , where the size of X is n_x and size of Y is n_y . The matrix size is $n_x \times n_y$; the row index is represented by i while the column index is represented by j , as shown in Fig. 10. The red arrows in the matrix represents the direction of the increment of the array index.

As illustrated in Fig. 10, when $j < n_x - i$, the array index falls into the top left part in the matrix. We denote

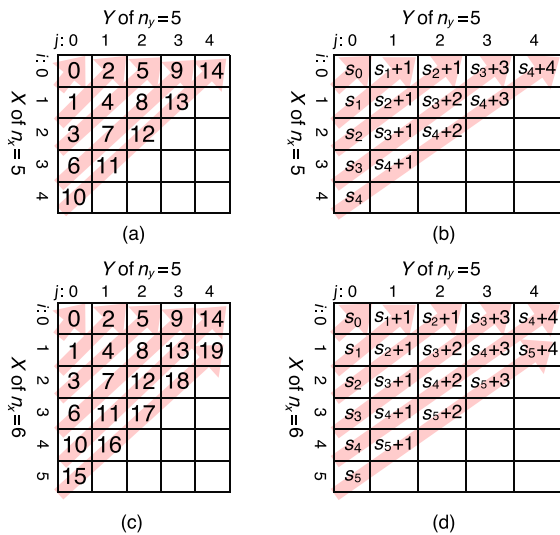


Fig. 10 Optimized memory access index — top left.

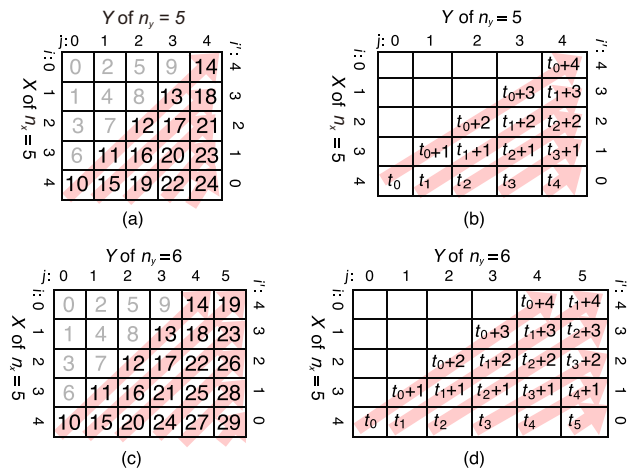


Fig. 11 Optimized memory access index — bottom right.

the index of an element in the left most column as S_i , where i is the row index of the element. As shown in Fig. 10b, the number increases along the red arrow from the left most column, e.g., $index(0, 1) = S_1 + 1$; $index(1, 3) = S_4 + 3$.

For an arbitrary element residing in the top left part of the matrix (i.e., $j < n_x - i$), if we denote the start position of the diagonal line that this element belongs to as $S_{i_{start}}$, then we can compute its index by the following formula:

$$index(i, j) = S_{i_{start}} + j \quad (8)$$

Because for any arbitrary element it forms a right triangle with x - and y -axes, the position of i_{start} can be found by counting the number of steps needed for this arbitrary element to reach the leftmost column:

$$i_{start} = i + j \quad (9)$$

Since the number of elements in each diagonal line increases in each line, we can use the arithmetic sequence to calculate $S_{i_{start}}$. However, as illustrated in Figs. 10c and 10d, when $n_x > n_y$, the computation has to be considered carefully. Since when the length of x -axis is greater than that of the y -axis, the arithmetic sequence stops at n_y and increases by n_y in each row, which can be formally defined as

$$i_{start} > n_y \Rightarrow i + j > n_y \quad (10)$$

Hence, we separate the computation in following two cases:

- Case: $i + j < n_y$

Recall that the sum of an arithmetic sequence can be calculated by

$$S_n = \frac{n(a_1 + a_n)}{2},$$

where S_n is the sum of n terms; a_1 is the first term; a_n is the n -th term.

As illustrated in Figs. 10a and 10b we get

$$S_{i_{start}} = 0 + 1 + 2 + \dots + i_{start} = \frac{i_{start} \times (i_{start} + 1)}{2} \quad (11)$$

Substituting Eq. (11) with Eqs. (8) and (9),

$$index(i, j) = S_{i_{start}} + j = \frac{(i + j) \times (i + j + 1)}{2} + j.$$

- Case: $i + j \geq n_y$

As illustrated in Figs. 10c and 10d, we get:

$$S_{i_{start}} = 0 + 1 + 2 + \dots + (n_y - 1) +$$

$$(i_{start} - n_y + 1) \times n_y = \frac{(n_y - 1) \times n_y}{2} + (i_{start} - n_y + 1) \times n_y \quad (12)$$

Substituting Eq. (12) with Eqs. (8) and (9),

$$index(i, j) = S_{i_{start}} + j = \frac{(ny - 1) \times ny}{2} + (i + j - ny + 1) \times ny + j.$$

We now considering the case when $j \geq n_x - i$, i.e., when the position falls into the bottom-right part of the matrix as shown in Fig. 11. Similar to the top-left case, two cases need to be considered.

We denote the cells (i, j) in the bottommost row (i.e., where the red arrows start from) as $T_{j_{start}}$. The $index(i, j)$ equals to $T_{j_{start}}$ adding the row index counting from the bottom (called the reverse row index). We denote the reverse row index by i' , which can be computed by the formula:

$$i' = n_x - 1 - i \quad (13)$$

Hence, we can calculate the $index(i, j)$ by

$$index(i, j) = T_{j_{start}} + i' = T_{j_{start}} + n_x - 1 - i \quad (14)$$

Similar to the Top-left case, the j_{start} is the number of steps required to reach the left most column from the current row:

$$j_{start} = j - i' = j - n_x + 1 + i \quad (15)$$

The arithmetic sequence is used to calculate the $T_{j_{start}}$. In this case, the sequence is in the decreasing order from n_y to $n_y - 1 - j_{start}$. Hence, the arithmetic sequence starts at the point when

$$j_{start} > n_y - n_x \Rightarrow j - n_x + 1 + i > n_y - n_x \quad (16)$$

When $n_y > n_x$, the computation needs to be addressed differently. Hence, we handle the calculation in following 3 cases:

- Case: $n_y \leq n_x$

$$T_{j_{start}} = index(n_x - 1, 0) + (n_y + (n_y - 1) + \dots + (n_y - 1 - j_{start})) = index(n_x - 1, 0) + \frac{(n_y + n_y - 1 - j_{start}) \times (j_{start})}{2} \quad (17)$$

Substituting Eqs. (14) and (15),

$$index(i, j) = index(n_x - 1, 0) + \frac{(2n_y - j + n_x - i) \times (j - n_x + 1 + i)}{2} + n_x - 1 - i \quad (18)$$

- Case: $n_y > n_x$ and $j_{start} \leq n_y - n_x$

In this case, the $T_{j_{start}}$ is defined as

$$T_{j_{start}} = index(n_x - 1, 0) + n_x \times j_{start} \quad (19)$$

By substituting Eqs. (14) and (15), we can compute

$index(i, j)$ as

$$index(i, j) = T_{j_{start}} + n_x - 1 - i = index(n_x - 1, 0) + n_x \times (j - n_x + 1 + i) + n_x - 1 - i \quad (20)$$

- Case: $n_y > n_x$ and $j_{start} > n_y - n_x$

In this case, we compute $T_{j_{start}}$ with the following formula:

$$T_{j_{start}} = index(n_x - 1, 0) + n_x(n_y - n_x) + (n_x + (n_x - 1) + \dots + (n_y - 1 - j_{start})) = index(n_x - 1, 0) + n_x(n_y - n_x) + \frac{(n_x + n_y - 1 - j_{start}) \times (j_{start} - (n_y - n_x))}{2} \quad (21)$$

Again, substituting $T_{j_{start}}$ into Eqs. (14) and (15), we get

$$index(i, j) = T_{j_{start}} + n_x - 1 - i = index(n_x - 1, 0) + n_x(n_y - n_x) + \frac{(n_y - j + i) \times (j + 1 + i - n_y)}{2} + n_x - 1 - i \quad (22)$$

6 Evaluation

To evaluate the effectiveness and efficiency of the proposed pattern mining algorithm, we have conducted extensive experiments with different time series dataset drawn from real life. We first apply the proposed algorithm on different datasets to evaluate the ability of the proposed pattern mining method in capturing the repeated patterns between two time series. Then we evaluate the performance of the serial implementation and the parallel implementation. Finally, we evaluate our proposed technique for memory optimization.

6.1 Preprocessing and parameters setting

We first evaluate the effectiveness of our proposed algorithm. In our experiments, the data preprocessing is needed before the pattern mining process. The preprocessing step includes removing trend, using Haar-wavelet-transform to smooth and reduce the complexity of time series with much noise and fluctuation. Preprocessing is beneficial to pattern mining, for example, the preprocessing steps such as Haar-wavelet-transform can reduce the size of the data, and hence will reduce the computation time.

In our experiments, we use the normalized cost threshold, offset threshold, and window threshold. Hence, the same parameters can be applied on different datasets. In normalized cost threshold, we use the

normalized cost function described in Section 3.3. Therefore, the costs are in the range between 0 to 1. The time-series data should also be normalized in the preprocessing step if they are in similar ranges. In normalized offset and window threshold, we represent the offset/window as a factor of the average length of the time series instead of actual size. Therefore, the threshold parameters are independent on the dataset.

6.2 Subsequence pattern mining

A number of experiments have been conducted to evaluate the quality of proposed DTW-based pattern mining algorithm. In this section, we evaluate the algorithm with different datasets. The datasets used in these experiments are from the real world, synthetic data, and random data.

6.2.1 Experiments with internet traffic data

Our first experiment is conducted by applying our algorithm on the datasets containing the same subsequence with different order. In this experiment, we use a subset of internet traffic data from Time Series Data Library (TSDL)^[30]. We crop a small piece of sequence, around 100 data points, as test dataset X . We also select another set of data with the similar length from the original dataset as dataset Y , and certain data in datasets X and Y are overlapped. In this experiment, we set the thresholds to the following values: $t = 0.13$, $o = 0.15$, $w = 0.2$. The DTW cumulative cost matrix and paths for discovered subsequences are visualized in Figs. 12 and 13, respectively.

The DTW cumulative cost matrix in Fig. 12 shows that the DTW calculation starts from the start of

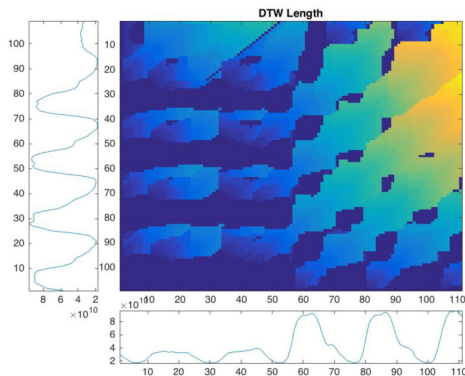


Fig. 12 Modified DTW method cumulative cost matrix. In two input data frames, the x -axis represents the length of input time series data (number of data points), the y -axis represents the value of each data point. The x - and y -axes in the Distance/Cost matrix represent the distance between two corresponding data points.

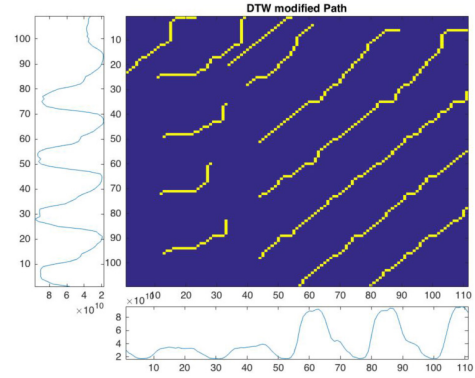


Fig. 13 Modified DTW method matched subsequence paths. In two input data frames, the x -axis represents the length of input time series data (number of data points), the y -axis represents the value of each data point. The x - and y -axes in the matrix represent the distance between two corresponding data points. The matching subsequences between two data frames are shown with the yellow line.

potential matching subsequence, which is the low cost regions in the figure, the computation continues as long as the subsequence is still a eligible candidate. Figure 13 shows the results of trace back from the DTW cumulative cost matrix to report matching subsequence with the length more than the window threshold. These results demonstrate that the proposed algorithm is able to capture the subsequence patterns between the time series. The exact subsequence is discovered, along with other similar subsequences.

This experiment shows that the modified DTW method picks up the local alignment regions in the matrix first, then calculates the possible subsequence alignments path. However, executing the algorithm with different distance functions and thresholds will generate slightly different alignment results. Although the algorithm will find different alignments, these alignments will still correspond to the similar subsequence between two sequences.

6.2.2 Effect of different thresholds

The thresholds designed in the proposed algorithm can help the algorithm to discover the patterns and have impact on subsequence found by the algorithm. In this section, we study how the threshold values will affect the performance of proposed algorithm.

In these experiments, we use the same dataset from the previous section. This way, we can see the effect on the results with different thresholds. We first study the cost threshold t , which controls the toleration of the matching subsequence. In this experiment, we reduce

the threshold t from 0.13 to 0.05. The results are shown in Fig. 14.

The results show that by reducing the cost threshold, the subsequences found by the algorithm lie in the lower cumulative cost in the path. Therefore, these subsequences are more similar. On the other hand, with larger value t , the algorithm will produce more matching sequences as it will consider more candidates.

We then study the effect of threshold o . This threshold controls how much the path can be diverged from the perfect matching sequence path (i.e., a one-to-one correspondence in time warping path). If the offset threshold o is too big, the discovered path will include long tails as demonstrated in Fig. 15, which uses a larger

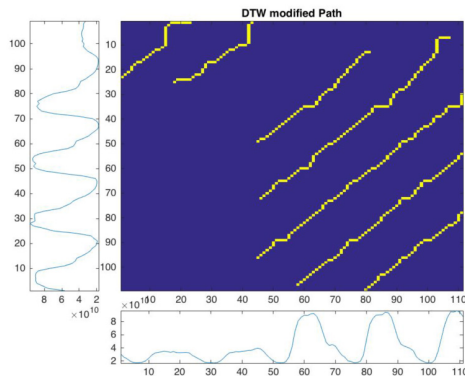


Fig. 14 Result with lower cost threshold. In two input data frames, the x-axis represents the length of input time series data (number of data points), the y-axis represents the value of each data point. The x- and y-axes in the matrix represent the distance between two corresponding data points. The matching subsequences between two data frames are shown with the yellow line.

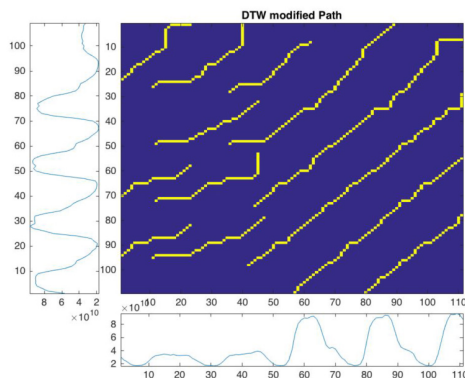


Fig. 15 Result with large offset threshold. In two input data frames, the x-axis represents the length of input time series data (number of data points), the y-axis represents the value of each data point. The x- and y-axes in the matrix represent the distance between two corresponding data points. The matching subsequences between two data frames are shown with the yellow line.

offset threshold of 1. On the other hand, if the offset is too small, the result may only include the straight path and they may be clustered together. Unlike other thresholds, the offset threshold o has a bigger influence on the quality of the result.

The window threshold is used to filter out the shorter sequences and only report the matching sequences that have the warping path longer than the threshold. In this experiment, we increase the window threshold from 0.2 to 0.3 and plot the results in Fig. 16. From this figure, we can see that by increasing the window length, the results produced by the algorithm is less than the small threshold.

6.2.3 Experiments with the data containing spikes — WordsSynonyms

We now evaluate how the proposed algorithm performs when the datasets are not perfect. In real-world time series data, peaks or spikes can be observed frequently. Thus it is essential to know how our algorithm can handle this problem. In these experiments, we use the WordSynonyms time series data from UCR Time Series Classification Archive^[31], which contains the possible patterns but also contains the spikes in one of the sequences. In this dataset, the training and testing datasets are different but are from the same source.

We compare the training and testing data from this dataset to evaluate our pattern mining algorithm. Unlike the training data, the testing dataset contains spikes in it. In order to see how well our algorithm performs, we compare the results produced with the original dataset against the dataset with the spikes being removed.

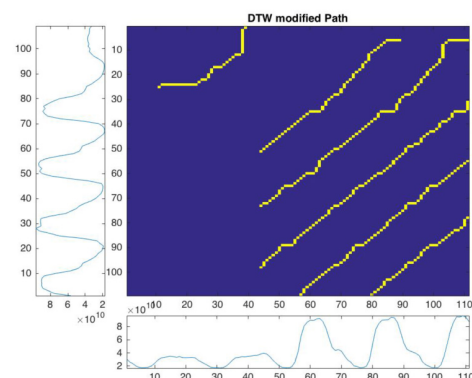


Fig. 16 Result with larger window threshold. In two input data frames, the x-axis represents the length of input time series data (number of data points), the y-axis represents the value of each data point. The x- and y-axes in the matrix represent the distance between two corresponding data points. The matching subsequences between two data frames are shown with the yellow line.

There are many different techniques to remove the spikes from the dataset. In our experiment, we choose Median filter to eliminate the spikes in time series dataset due to the popularity of this methods. The experiment results are shown in Figs. 17 and 18.

From these figures, we can see that the matching subsequence results found in the original data are similar to the result obtained with the data in which the spikes are removed. These results prove that our method is resistant to spikes in the data and it will treat them as part of the pattern.

6.2.4 Experiments with the circular trend data

In this section we evaluate the performance of proposed algorithm with the data that have the circular patterns.

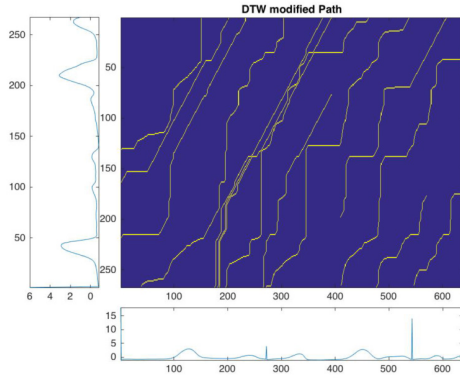


Fig. 17 Result of WordsSynonyms data. In two input data frames, the x-axis represents the length of input time series data (number of data points), the y-axis represents the value of each data point. The x- and y-axes in the matrix represent the distance between two corresponding data points. The matching subsequences between two data frames are shown with the yellow line.

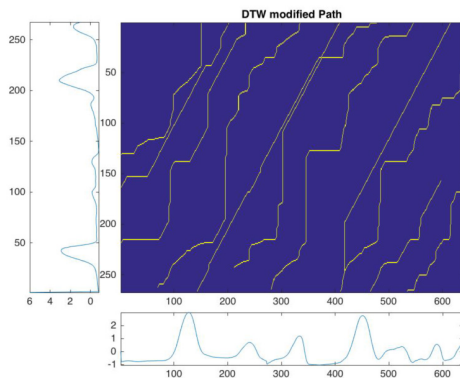


Fig. 18 Result of WordsSynonyms data filtered. In two input data frames, the x-axis represents the length of input time series data (number of data points), the y-axis represents the value of each data point. The x- and y-axes in the matrix represent the distance between two corresponding data points. The matching subsequences between two data frames are shown with the yellow line.

In this experiment, we use the data with the repeating patterns and the increasing trend (the absolute value of the data increases over time) from TSDL^[30]. The time series is tested against itself in this experiment to see how our method handles the type of time series data. We create another dataset by removing the trend in the data, and compare the results with the original dataset. As we can see from Figs. 19 and 20, our algorithm is able to pick up all the circular patterns after removing the trend.

6.2.5 Experiments with noisy data — cluster load

We then evaluate our algorithm using the data with the noisy signals. In this experiment, we use two sets of trace data in clusters from Ref. [32]. The

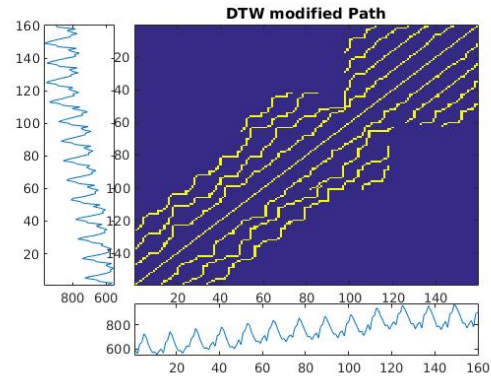


Fig. 19 Data with circular trend. In two input data frames, the x-axis represents the length of input time series data (number of data points), the y-axis represents the value of each data point. The x- and y-axes in the matrix represent the distance between two corresponding data points. The matching subsequences between two data frames are shown with the yellow line.

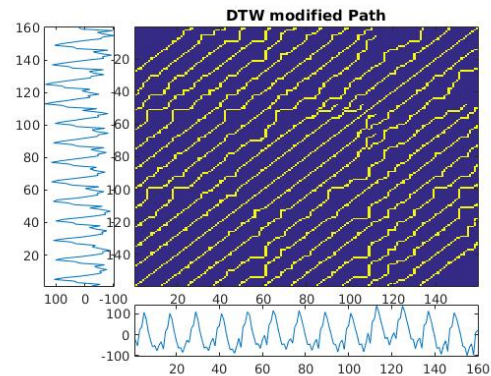


Fig. 20 Data with circular trend—removed trend. In two input data frames, the x-axis represents the length of input time series data (number of data points), the y-axis represents the value of each data point. The x- and y-axes in the matrix represent the distance between two corresponding data points. The matching subsequences between two data frames are shown with the yellow line.

result alignments look very scattered without any preprocessing. To improve the result, we applied the Haar Wavelet transform^[33] to the data sequence. Figures 21 and 22 show the result before and after applying the Haar-Wavelet-transform to the two sets of data. Figures 23 and 24 demonstrate that the paths of the patterns in the cost matrix. The results show that the patterns are discovered correctly.

6.3 Performance

In this section, we conduct the performance analysis on both serial and parallel implementation. In these experiments, we use a very large dataset collected by Ref. [17]. We test the performance by using different implementations to analyze a subset of the data with

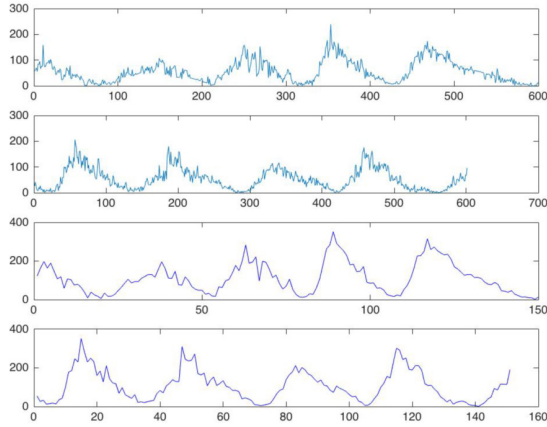


Fig. 21 Haar wavelet transform on complicated time series. In these figures, the x-axis represents the length of input time series data (number of data points), the y-axis represents the value of each data point.

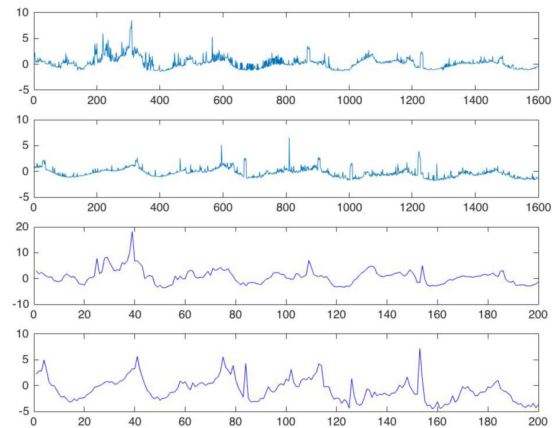


Fig. 22 Haar wavelet transform on complicated time series. In these figures, the x-axis represents the length of input time series data (number of data points), the y-axis represents the value of each data point.

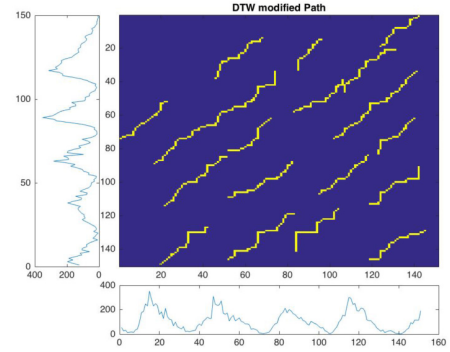


Fig. 23 Modified DTW on transformed sequence. In two input data frames, the x-axis represents the length of input time series data (number of data points), the y-axis represents the value of each data point. The x- and y-axes in the matrix represent the distance between two corresponding data points. The matching subsequences between two data frames are shown with the yellow line.

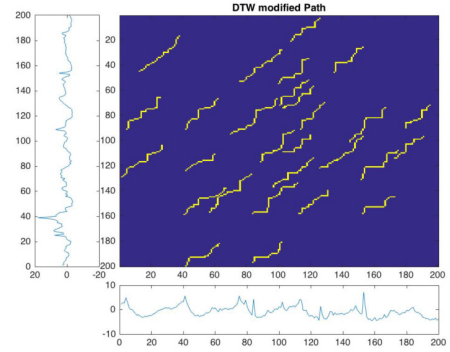


Fig. 24 Modified DTW on transformed sequence. In two input data frames, the x-axis represents the length of input time series data (number of data points), the y-axis represents the value of each data point. The x- and y-axes in the matrix represent the distance between two corresponding data points. The matching subsequences between two data frames are shown with the yellow line.

different lengths. The experiments are conducted on a machine with NVIDIA Tesla K40 GPU with 12 GB memory. We use a subset from the dataset that can fit into the GPU memory.

We profile the implementation using the same data on the vertical and horizontal axes. The execution time is recorded with different lengths of the data. Figure 25 shows the execution time of the serial implementation (Host_Matrix) and the CUDA implementation (CUDA_Matrix).

From the figure, we can see that the CUDA implementation achieves the faster execution time than the serial implementation.

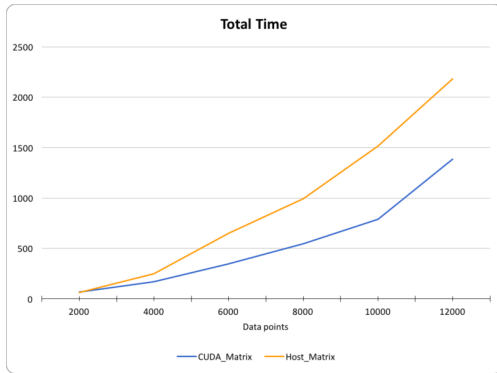


Fig. 25 Execution time of the serial and CUDA implementations. The *x*-axis represents the length of input time series data (number of data points), the *y*-axis represents the execution time in millisecond.

We then compare the execution time between the CUDA implementation and the CUDA implementation with the optimised memory access.

It can be seen from Fig. 26 that by optimizing the memory access the optimized CUDA implementation has the shorter execution time. This result demonstrates the effectiveness of our technique in optimizing the memory access.

7 Conclusion

In this paper, we proposed a novel algorithm to find all similar subsequences in the time series data. Our algorithm first identifies the possible start positions of similar alignments, and then uses the dynamic programming technique to expand the subsequences. Further, we parallelize the proposed algorithm using GPGPU and CUDA, and develop a optimization technique to improve the memory access in the GPU



Fig. 26 Comparison of DTWM calculation time. The *x*-axis represents the length of input time series data (number of data points), the *y*-axis represents the execution time in millisecond.

implementation. Extensive evaluation experiments have been conducted. The results show that our algorithm can discover similar subsequences between two time series and achieve much better performance with the GPU parallelization.

Acknowledgment

This research was supported by the National Natural Science Foundation of China (No. 61602215), the Science Foundation of Jiangsu Province (No. BK20150527), the EU Horizon 2020 — Marie Skłodowska-Curie Actions through the project entitled Computer Vision Enabled Multimedia Forensics and People Identification (Project No. 690907, Acronym: IDENTITY).

References

- [1] P. Esling and C. Agon, Time-series data mining, *ACM Comput. Surv.*, vol. 45, no. 1, p. 12, 2012.
- [2] C. T. Li, Y. Y. Yuan, and R. Wilson, An unsupervised conditional random fields approach for clustering gene expression time series, *Bioinformatics*, vol. 24, no. 21, pp. 2467–2473, 2008.
- [3] K. P. Chan and A. W. C. Fu, Efficient time series matching by wavelets, in *Proc. 15th Int. Conf. Data Engineering*, Sydney, Australia, 1999, pp. 126–133.
- [4] X. P. Ge and P. Smyth, Deformable Markov model templates for time-series pattern matching, in *Proc. 6th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, Boston, MA, USA, 2000, pp. 81–90.
- [5] K. Kalpakis, D. Gada, and V. Puttagunta, Distance measures for effective clustering of ARIMA time-series, in *Proc. 2001 IEEE Int. Conf. Data Mining*, San Jose, CA, USA, 2001, pp. 273–280.
- [6] J. Lin, E. Keogh, S. Lonardi, and P. Patel, Finding motifs in time series, in *Proc. 2nd Workshop on Temporal Data Mining*, Edmonton, Canada, 2002, pp. 53–68.
- [7] B. Chiu, E. Keogh, and S. Lonardi, Probabilistic discovery of time series motifs, in *Proceedings of the 9th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, Washington, DC, USA, 2003, pp. 493–498.
- [8] G. Das, K. I. Lin, H. Mannila, G. Renganathan, and P. Smyth, Rule discovery from time series, in *Proc. 4th Int. Conf. Knowledge Discovery and Data Mining*, Menlo Park, CA, USA, 1998, pp. 16–22.
- [9] F. Höppner, Discovery of temporal patterns, in *Principles of Data Mining and Knowledge Discovery*, L. De Raedt and A. Siebes, eds. Springer, 2001, pp. 192–203.
- [10] E. J. Keogh and M. J. Pazzani, An enhanced representation of time series which allows fast and accurate classification, clustering and relevance feedback, in *Proc. 4th Int. Conf. Knowledge Discovery and Data Mining*, New York, NY, USA, 1998, pp. 239–243.
- [11] M. Hegland, W. Clarke, and M. Kahn, Mining the MACHO dataset, *Comput. Phys. Commun.*, vol. 142, nos. 1–3, pp. 22–28, 2001.

- [12] D. Dasgupta and S. Forrest, Novelty detection in time series data using ideas from immunology, in *Proc. 8th Int. Conf. Intelligent Systems*, Denver, CO, USA, 1999, pp. 82–87.
- [13] J. W. Han, G. Z. Dong, and Y. W. Yin, Efficient mining of partial periodic patterns in time series database, in *Proc. 15th Int. Conf. Data Engineering*, Sydney, Australia, 1999, pp. 106–115.
- [14] M. Müller, *Information Retrieval for Music and Motion*. Springer, 2007.
- [15] C. Faloutsos, M. Ranganathan, and Y. Manolopoulos, Fast subsequence matching in time-series databases, *ACM SIGMOD Rec.*, vol. 23, no. 2, pp. 419–429, 1994.
- [16] Y. Sakurai, C. Faloutsos, and M. Yamamuro, Stream monitoring under the time warping distance, in *Proc. 23rd Int. Conf. Data Engineering*, Istanbul, Turkey, 2007, pp. 1046–1055.
- [17] T. Rakthanmanon, B. Campana, A. Mueen, G. Batista, B. Westover, Q. Zhu, J. Zakaria, and E. Keogh, Searching and mining trillions of time series subsequences under dynamic time warping, in *Proc. 18th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, Beijing, China, 2012, pp. 262–270.
- [18] X. J. Wei, C. T. Li, Z. Lei, D. Yi, and S. Z. Li, Dynamic image-to-class warping for occluded face recognition, *IEEE Transactions on Information Forensics and Security*, vol. 9, no. 12, pp. 2035–2050, 2014.
- [19] M. Toyoda and Y. Sakurai, Discovery of cross-similarity in data streams, in *Proc. 26th Int. Conf. Data Engineering*, Long Beach, CA, USA, 2010, pp. 101–104.
- [20] T. Nishii, T. Hiroyasu, M. Yoshimi, M. Miki, and H. Yokouchi, Similar subsequence retrieval from two time series data using homology search, in *Proc. 2010 IEEE Int. Conf. Systems Man and Cybernetics*, Istanbul, Turkey, 2010, pp. 1062–1067.
- [21] L. M. Xiao, Y. Zheng, W. Q. Tang, G. C. Yao, and L. Ruan, Parallelizing dynamic time warping algorithm using prefix computations on GPU, in *Proc. 10th Int. Conf. High Performance Computing and Communications & 2013 IEEE Int. Conf. Embedded and Ubiquitous Computing*, Zhangjiajie, China, 2013, pp. 294–299.
- [22] D. Sart, A. Mueen, W. Najjar, E. Keogh, and V. Niennattrakul, Accelerating dynamic time warping subsequence search with GPUs and FPGAs, in *Proc. 10th Int. Conf. Data Mining*, Sydney, Australia, 2010, pp. 1001–1006.
- [23] Y. D. Zhang, K. Adl, and J. Glass, Fast spoken query detection using lower-bound dynamic time warping on graphical processing units, in *Proc. 2012 IEEE Int. Conf. Acoustics, Speech and Signal Processing*, Kyoto, Japan, 2012, pp. 5173–5176.
- [24] A. Vahdatpour, N. Amini, and M. Sarrafzadeh, Toward unsupervised activity discovery using multi-dimensional motif detection in time series, in *Proc. 21st Int. Joint Conf. Artificial Intelligence*, Pasadena, CA, USA, 2009, pp. 1261–1266.
- [25] A. Mueen, E. J. Keogh, Q. Zhu, S. Cash, and M. B. Westover, Exact discovery of time series motifs, in *2009 SIAM Int. Conf. Data Mining*, Sparks, NV, USA, 2009, pp. 473–484.
- [26] M. Toyoda, Y. Sakurai, and Y. Ishikawa, Pattern discovery in data streams under the time warping distance, *VLDB J.*, vol. 22, no. 3, pp. 295–318, 2013.
- [27] S. Srikanthan, A. Kumar, and R. Gupta, Implementing the dynamic time warping algorithm in multithreaded environments for real time and unsupervised pattern discovery, in *Proc. 2nd Int. Conf. Computer and Communication Technology*, Allahabad, India, 2011, pp. 394–398.
- [28] N. Takahashi, T. Yoshihisa, Y. Sakurai, and M. Kanazawa, A parallelized data stream processing system using dynamic time warping distance, in *Proc. 2009 Int. Conf. Complex, Intelligent and Software Intensive Systems*, Fukuoka, Japan, 2009, pp. 1100–1105.
- [29] NVIDIA Corporation, *NVIDIA CUDA C Programming Guide*, 2011.
- [30] R. J. Hyndman, Time series data library, <https://robjhyndman.com/TSDL/>, 2016.
- [31] Y. P. Chen, E. Keogh, B. Hu, N. Begum, A. Bagnall, A. Mueen, and G. Batista, The UCR time series classification archive, www.cs.ucr.edu/~eamonn/time-series.data/, 2015.
- [32] C. Reiss, J. Wilkes, and J. L. Hellerstein, Google cluster-usage traces: Format + Schema, Technical report, Google Inc., Mountain View, CA, USA, November 2011, revised 2014-11-17 for version 2.1, <https://github.com/google/cluster-data>.
- [33] R. S. Stankovic and B. J. Falkowski, The Haar wavelet transform: Its status and achievements, *Comput. Electr. Eng.*, vol. 29, no. 1, pp. 25–44, 2003.



Zhuoer Gu acquired the BSc degree from Shanxi University in 2013, and in 2018 he completed his PhD in the University of Warwick. He has focused on time series data mining for years. His research interests cover time series analysis and data mining, machine learning, and high performance computing.



Haiming Zhao received the BEng degree in computing from Imperial College London and MSc degree in data analytics from University of Warwick, in 2015 and 2016, respectively. Since then, she has been working as a software engineer in the finance industry.



Huanzhou Zhu received the bachelor degree in information system engineering from Imperial College London, UK, in 2008, and received the master and PhD degrees in computer science from the University of Warwick, UK, in 2009 and 2016 respectively. He is now a post-doctoral researcher at the Imperial College

London. His areas of interest are big data processing systems, parallel and distributed computation systems, high performance computing, cloud computing, and computer security.



Keyang Cheng is a member of CCF. He received the PhD degree from Nanjing University of Aeronautics & Astronautics in 2015. He has co-authored more than 30 journal and conference papers. He was at University of Warwick, UK, as a post-doctoral researcher in 2016. He is currently

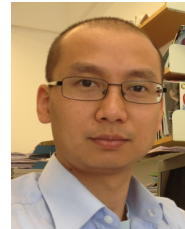
an assistant professor and researcher in the School of Computer Science and Telecommunications Engineering of Jiangsu University. His current research interests lie in the areas of pattern recognition, computational intelligence, and computer vision.



Chang-Tsun Li received the MSc degree in computer science from U.S. Naval Postgraduate School, USA, in 1992, and the PhD degree in computer science from the University of Warwick, UK, in 1998. He was a visiting professor of the Department of Computer Science at U.S. Naval Postgraduate School in the second

half of 2001. He was a professor of the Department of Computer Science at the University of Warwick, UK, until

Dec 2016. He is currently a professor of the School of Computing and Mathematics, Charles Sturt University, Australia, leading the Data Science Research Unit. His research interests include multimedia forensics and security, biometrics, data mining, machine learning, data analytics, computer vision, image processing, pattern recognition, bioinformatics, and content-based image retrieval. The outcomes of his multimedia forensics and machine learning research have been translated into award-winning commercial products protected by a series of international patents and have been used by a number of police forces and courts of law around the world. He is currently an Associate Editor of the EURASIP Journal of Image and Video Processing (JIVP) and Associate Editor of IET Biometrics. He involved in the organisation of many international conferences and workshops and also served as member of the international program committees for several international conferences. He is also actively contributing keynote speeches and talks at various international events.



Ligang He received the bachelor and master degrees from the Huazhong University of Science and Technology, China, in 1998 and 2001, respectively. He received the PhD degree in computer science from the University of Warwick, UK, in 2006. He was also a post-doctoral researcher at the University of Cambridge,

UK. In 2006, he joined the Department of Computer Science at the University of Warwick as an assistant professor, and then became an associate professor. He has published more than 100 papers in journals (such as IEEE TPDS, ACM Computing Surveys, JPDC, JCSS) and conferences (such as IPDPS, ICPP, ICWS, VLDB). His areas of interest are parallel and distributed computing, high performance computing, and big data analysis.